

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN
— o0o —



ĐỒ ÁN LẬP TRÌNH TÍNH TOÁN

**XÂY DỰNG CHƯƠNG TRÌNH TÌM ĐƯỜNG ĐI
NGẮN NHẤT
TRÊN ĐỒ THỊ CÓ HƯỚNG CÓ TRỌNG SỐ**

Người hướng dẫn:	ThS. TRẦN HỒ THỦY TIÊN	
Sinh viên thực hiện:	Trần Văn Trường Vũ	LỚP: 25Nh11
	Trần Ngọc Bảo Quyên	LỚP: 25Nh11

Đà Nẵng, 06/2026

MỤC LỤC

Mục lục	i
Danh sách hình vẽ	ii
MỞ ĐẦU	1
1 TỔNG QUAN ĐỀ TÀI	2
1.1 Lý thuyết đồ thị có hướng (Directed Graph / Digraph)	2
1.2 Bài toán tìm đường đi ngắn nhất (Shortest Path Problem)	2
1.3 Thuật toán Dijkstra	2
1.4 Đồ họa máy tính và mô phỏng trực quan	3
2 CƠ SỞ LÝ THUYẾT	4
2.1 Ý tưởng xây dựng ứng dụng	4
2.2 Cơ sở lý thuyết thuật toán và đồ họa	4
2.2.1 Thuật toán đường đi ngắn nhất Dijkstra	4
2.2.2 Lý thuyết đồ họa và tính toán tọa độ vector hình học	4
3 TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN	7
3.1 Phát biểu bài toán hệ thống	7
3.2 Tổ chức cấu trúc dữ liệu cụ thể	7
3.3 Thiết kế máy trạng thái thuật toán mô phỏng	7
4 CHƯƠNG TRÌNH VÀ KẾT QUẢ	8
4.1 Tổ chức mô hình chương trình	8
4.2 Ngôn ngữ cài đặt	8
4.3 Kết quả thực thi chương trình	8
4.3.1 Giao diện trang thông tin đề án ban đầu	8
4.3.2 Mô phỏng đồ họa động tương tác trực quan	9
4.3.3 Nhận xét đánh giá hệ thống	9
5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	11
5.1 Kết luận	11
5.2 Hướng phát triển đề tài tương lai	11
PHỤ LỤC	13

DANH SÁCH HÌNH VẼ

4.1	Giao diện chính giới thiệu thông tin đồ án trên màn hình Console	8
4.2	Giao diện đồ họa phân bố hệ thống các đỉnh theo quỹ đạo tròn đồng tâm .	9
4.3	Bảng điều khiển nhật ký hoạt động cập nhật trạng thái loang nhãn từng bước	9
4.4	Biểu diễn lộ trình ngắn nhất bằng hiệu ứng đổi màu cung liên kết sang sắc đỏ	9

MỞ ĐẦU

1. Mục đích thực hiện đề tài

Trong thời đại khoa học và công nghệ bùng nổ hiện nay, nhu cầu ứng dụng công nghệ thông tin vào bài toán thực tế ngày càng tăng cao. Phần mềm hỗ trợ trực quan hóa giúp công việc thiết kế và quản trị trở nên tối ưu và tự động hóa dễ dàng hơn. Đề tài là cơ sở giúp nhóm rèn luyện tư duy thiết kế giải thuật hợp lý, áp dụng các kiến thức cốt lõi đã học trong học phần Toán rời rạc, Kỹ thuật lập trình và Cấu trúc dữ liệu nhằm giải quyết các bài toán kỹ thuật phức tạp. Qua đồ án này, nhóm hướng tới việc nắm bắt sâu sắc bản chất của thuật toán Dijkstra trên cấu trúc đồ thị có hướng, đồng thời nâng cao tư duy xây dựng ứng dụng tương tác thời gian thực.

2. Mục tiêu đề tài

Mục tiêu chính của đồ án này là phân tích chuyên sâu, thiết kế và thực thi ứng dụng tìm đường đi ngắn nhất giữa hai điểm bất kỳ được chọn trên mô hình đồ thị có hướng có trọng số.

- Tiếp nhận ma trận kề từ tệp tin dữ liệu có cấu trúc đầu vào hoặc thông qua cơ chế tùy chọn linh hoạt.
- Áp dụng chính xác thuật toán Dijkstra để tính toán nhân đường đi tối ưu và thực hiện truy vết lộ trình.
- Ứng dụng thư viện đồ họa chuyên dụng Raylib để thiết kế giao diện máy trạng thái mô phỏng động, biểu diễn quá trình duyệt đỉnh tối ưu và cập nhật chi phí một cách trực quan *Step-by-Step*.

3. Phạm vi và đối tượng nghiên cứu

- **Phạm vi nghiên cứu:** Vận dụng tổng hợp các kiến thức nền tảng về Toán rời rạc hệ đồ thị, kỹ thuật xử lý luồng sự kiện đồ họa thời gian thực, thuật toán tối ưu hóa nhân và quản lý cấu trúc dữ liệu tĩnh trong ngôn ngữ lập trình.
- **Đối tượng nghiên cứu:** Cơ chế thao tác tệp cấu trúc hệ thống, giải thuật tính toán hình học không gian (xác định vị trí đỉnh vòng tròn, vector hướng góc mũi tên phụ thuộc), cấu trúc dữ liệu ma trận kề trọng số và kỹ thuật xử lý phân tách đa trạng thái (*State Machine*).

4. Phương pháp nghiên cứu

Nhóm tập trung nghiên cứu hệ thống giáo trình, tài liệu chuyên ngành về thiết kế giải thuật toán tối ưu kết hợp phân tích mã nguồn đồ họa ứng dụng. Thực hiện khảo sát, đánh giá hiệu năng trực quan giữa các trạng thái kiểm thử dữ liệu thực tế, đồng thời tham khảo các tài liệu lập trình đồ họa tương tác trên Internet để tối ưu hóa trải nghiệm tương tác giao diện người dùng trên màn hình đồ họa.

1. TỔNG QUAN ĐỀ TÀI

Hệ thống tập trung nghiên cứu và xây dựng chương trình giải quyết bài toán cốt lõi trên mô hình lý thuyết đồ thị kết hợp kỹ thuật kết xuất máy tính. Các nền tảng lý thuyết cấu thành đề tài bao gồm:

1.1. Lý thuyết đồ thị có hướng (Directed Graph / Digraph)

Đồ thị có hướng là một cấu trúc toán học được định nghĩa bằng bộ cặp vô hướng $G = (V, E)$, trong đó V là tập hợp các nút gọi là đỉnh (Vertices), và E là tập hợp các cung gọi là cạnh có hướng (Directed Edges) [1, 2]. Mỗi cung thuộc tập E được biểu diễn dưới dạng một cặp đỉnh có thứ tự (u, v) , xác định hướng đi một chiều từ đỉnh gốc u đến đỉnh ngọn v . Trong các bài toán thực tiễn, đồ thị có hướng mô hình hóa hoàn hảo các hệ thống bất đối xứng như dòng luân chuyển giao thông đường một chiều, sơ đồ quan hệ phụ thuộc giữa các tiến trình công việc, hoặc kiến trúc liên kết điều hướng dữ liệu trên không gian Internet toàn cầu. Khi mỗi cung (u, v) được gán kèm một giá trị số thực $W(u, v) > 0$, mô hình trở thành đồ thị có hướng có trọng số, biểu thị cho chi phí, khoảng cách vật lý hoặc lượng thời gian tiêu hao để dịch chuyển qua lại giữa các nút hạ tầng.

1.2. Bài toán tìm đường đi ngắn nhất (Shortest Path Problem)

Bài toán tìm đường đi ngắn nhất là bài toán tìm một quỹ đạo di chuyển nối liền giữa một đỉnh nguồn gốc u_{start} cho tới một đỉnh đích v_{target} trên không gian mạng lưới liên kết sao cho tổng chi phí trọng số của tất cả các cung cấu thành nên lộ trình đó đạt giá trị cực tiểu. Đây là một bài toán tối ưu hóa tổ hợp kinh điển có phổ ứng dụng thực tiễn vô cùng rộng lớn. Lời giải chính xác của bài toán này chính là lõi thuật toán vận hành cho các hệ thống phần mềm định vị vệ tinh toàn cầu (GPS), hệ thống quản trị bài toán phân phối Logistic chuỗi cung ứng, hệ quy hoạch mạch tích hợp bán dẫn trên bo mạch điện tử, và các cơ chế định tuyến, phân tách luồng gói tin truyền tải trên hạ tầng mạng máy tính.

1.3. Thuật toán Dijkstra

Thuật toán Dijkstra được phát minh bởi nhà khoa học máy tính Edsger W. Dijkstra vào năm 1956 và công bố chính thức vào năm 1959 [3]. Đây là giải thuật tối ưu hóa tham lam (Greedy) kinh điển dùng để tìm đường đi ngắn nhất từ một đỉnh nguồn đơn lẻ đến một hoặc toàn bộ các đỉnh còn lại trên mô hình đồ thị có hướng có trọng số với điều kiện tất cả các cung đều có giá trị chi phí không âm ($W(u, v) \geq 0$). Nguyên lý cốt lõi của giải thuật dựa trên việc duy trì và loang một tập các nhãn khoảng cách tạm thời. Tại mỗi bước lặp, thuật toán trích xuất ra một đỉnh có giá trị khoảng cách ngắn nhất hiện tại để cố định nhãn, sau đó tiến hành quá trình tối ưu hóa (Relaxation) để hạ thấp chi phí nhãn khoảng cách của toàn bộ các nút lân cận kề sau nó. Do áp dụng chiến lược tham lam, thuật toán luôn đảm bảo tính chính xác và đạt tới nghiệm tối ưu toàn cục mà không cần phải duyệt vét cạn toàn bộ không gian trạng thái.

1.4. Đồ họa máy tính và mô phỏng trực quan

Đồ họa máy tính đóng vai trò thiết yếu trong việc hiện đại hóa cách thức tiếp cận hệ thống tri thức giải thuật trừu tượng [4]. Thay vì chỉ hiển thị các kết quả tính toán cuối cùng dưới dạng văn bản (Console) tĩnh lập lờ, việc ứng dụng kỹ thuật mô phỏng hình ảnh động thời gian thực cho phép người dùng trực tiếp quan sát toàn bộ cơ chế vận hành bên trong của thuật toán. Thông qua các đối tượng đồ họa trực quan (như khối tròn biểu diễn đỉnh, cung vẽ bằng mũi tên hình học kết hợp hộp thoại trọng số), hệ thống đồ họa sẽ tái hiện sinh động tiến trình loang nhãn. Bằng cách thay đổi linh hoạt các gam màu trạng thái của hệ thống kết xuất (sắc vàng cho phần tử đang duyệt, sắc xanh cho các đỉnh đã tối ưu hoàn toàn, sắc đỏ cho lộ trình cuối cùng), tiến trình hoạt động nội tại của cấu trúc dữ liệu đồ thị sẽ được phơi bày trực quan từng bước (*Step-by-Step*), giúp tối ưu hóa hiệu quả học tập và thử nghiệm.

2. CƠ SỞ LÝ THUYẾT

2.1. Ý tưởng xây dựng ứng dụng

Để xử lý trọn vẹn yêu cầu đề tài, hệ thống xây dựng mô hình tách biệt giữa pha điều hướng dữ liệu đầu vào và pha xử lý đồ họa tuần tự. Chương trình vận dụng kỹ thuật cấu trúc hóa luồng đọc dữ liệu đầu vào từ tệp tin văn bản hoặc điều phối cấu trúc tĩnh kết hợp đồng bộ cơ chế thời gian trễ tự nhiên giúp đồng bộ vận tốc thực thi thuật toán ăn khớp với tần số quét của màn hình hiển thị.

2.2. Cơ sở lý thuyết thuật toán và đồ họa

2.2.1. Thuật toán đường đi ngắn nhất Dijkstra

Gọi $G = (V, E)$ là đồ thị có hướng có trọng số. Hàm trọng số $W(u, v) \geq 0$ xác định chi phí đi từ đỉnh u đến đỉnh v . Thuật toán duy trì tập các đỉnh T đại diện cho tập các nút chưa cố định nhãn tối ưu. Tại mỗi bước lặp, thuật toán thực hiện:

1. Chọn đỉnh $u \in T$ sao cho nhãn khoảng cách đạt cực tiểu:

$$L[u] = \min_{i \in T} \{L[i]\} \quad (2.1)$$

2. Loại bỏ u ra khỏi tập nhãn tạm thời: $T \leftarrow T \setminus \{u\}$.
3. Với mỗi đỉnh v kề sau đỉnh u và $v \in T$, thực hiện cập nhật tối ưu (Relaxation):

$$\text{Nếu } L[u] + W(u, v) < L[v] \implies L[v] = L[u] + W(u, v), \quad \text{par}[v] = u \quad (2.2)$$

2.2.2. Lý thuyết đồ họa và tính toán tọa độ vector hình học

Để biểu diễn toàn vẹn một đồ thị có hướng lên không gian màn hình hai chiều tương tác, hệ thống cần xử lý bài toán hình học tính toán một cách tuần tự từ pha phân bố vị trí đỉnh, pha thiết lập đường thẳng liên kết cho đến pha dựng mũi tên hướng tiếp xúc ngoài biên hạt tròn. Tiến trình toán học được rã chi tiết như sau:

a) Tiến trình 1: Tính toán tọa độ và phân bố vị trí các đỉnh (Phần Đầu)

Khi bắt đầu khởi tạo, hệ thống cần sắp xếp đồng đều n đỉnh thực tế lên không gian đồ họa nhằm tối ưu hóa góc nhìn trực quan, tránh hiện tượng các nút tròn nằm đè chồng chéo. Chương trình vận dụng hệ phương trình tham số của đường tròn trong hình học không gian phẳng Descartes để quy hoạch các đỉnh cố định theo một quỹ đạo elip/tròn đồng tâm liên tục:

$$x_i = x_0 + R_{\text{layout}} \cdot \cos(\alpha_i) \quad (2.3)$$

$$y_i = y_0 + R_{\text{layout}} \cdot \sin(\alpha_i) \quad (2.4)$$

Trong kiến trúc cấu trúc mảng của ứng dụng, tâm quy hoạch hình học được chọn tại tọa độ cố định $I(x_0, y_0) = (430, 400)$ và bán kính vòng tròn phân bố là $R_{\text{layout}} = 280$ pixel.

Để phân chia đều khoảng cách các đỉnh trên vòng xoay toàn phần 2π Radian, góc định vị góc xoay α_i của đỉnh thứ i (với cấu trúc chỉ số danh định i chạy tuần tiến từ $1 \rightarrow n$) được tính toán qua công thức:

$$\alpha_i = (i - 1) \cdot \frac{2\pi}{n} \quad (2.5)$$

Nhờ đó, toàn bộ hệ thống tọa độ mảng tĩnh của từng đỉnh được dàn đều cân đối, tạo điều kiện thuận lợi cho việc kết xuất đồ họa ở các phân lớp phía sau.

b) Tiến trình 2: Thiết lập đoạn thẳng liên kết nối tâm (Phần Thân)

Sau khi hạ tầng vị trí các nút tròn đã định vị vững chắc, một cung liên kết hướng xuất phát từ đỉnh nguồn $U(x_{\text{start}}, y_{\text{start}})$ di chuyển đến đỉnh đích $V(x_{\text{end}}, y_{\text{end}})$ ban đầu được xác định bởi một đoạn thẳng nối tuyến tính giữa tâm hai hình tròn. Tuy nhiên, nếu chỉ sử dụng nét thẳng đơn thuần để vẽ từ tâm sang tâm, nét vẽ này sẽ che khuất nhãn ký tự định danh của nút tròn, đồng thời làm mất đi không gian để đặt thành phần điều hướng. Do đó, hệ thống bắt buộc phải tính toán và trích xuất một góc chỉ phương θ chạy dọc theo cạnh nối nhằm xác định xu hướng dốc của đường thẳng liên kết.

c) Tiến trình 3: Tính toán điểm dừng biên và dựng mũi tên chỉ hướng (Phần Đuôi)

Đây là phân đoạn hình học xử lý cốt lõi: đầu nhọn của mũi tên hướng bắt buộc phải dừng lại và chạm vừa khít vào phần đường biên ngoài của vòng tròn nút đích V , tuyệt đối không được xâm lấn hay chui vào bên trong tâm.

- **Bước 1: Xác định góc nghiêng chỉ phương θ :** Sử dụng hàm toán học lượng giác ngược `atan2f` dựa trên độ sai lệch không gian $(\Delta y, \Delta x)$ giữa hai điểm tâm. Hàm này giúp xác định góc hướng θ (Radian) chuẩn xác trên mọi góc phần tư hình học và loại trừ triệt để lỗi chia cho số 0 khi hai nút nằm thẳng hàng dọc đứng:

$$\theta = \text{atan2}(y_{\text{end}} - y_{\text{start}}, x_{\text{end}} - x_{\text{start}}) \quad (2.6)$$

- **Bước 2: Tính tọa độ điểm tiếp xúc rìa ngoài đường tròn (Tip):** Mỗi hạt đỉnh trong chương trình có bán kính cố định là $r = 32$ pixel. Để đầu nhọn mũi tên chạm đúng biên ngoài (và cách ra một khoảng hở 5 pixel giúp sơ đồ thoáng đãng hơn), điểm dừng $\text{tip}(x_{\text{tip}}, y_{\text{tip}})$ phải lùi ngược hướng từ tâm nút đích V về phía nút nguồn U một khoảng cách bằng $r + 5 = 37$ pixel. Công thức lượng giác xác định điểm này là:

$$x_{\text{tip}} = x_{\text{end}} - 37 \cdot \cos(\theta) \quad (2.7)$$

$$y_{\text{tip}} = y_{\text{end}} - 37 \cdot \sin(\theta) \quad (2.8)$$

- **Bước 3: Xác định tọa độ hai cánh tam giác mũi tên:** Để tạo hình cấu trúc tam giác bẹt thể hiện tính hướng cho cung liên kết, chương trình cần định vị thêm tọa độ của hai điểm chân cánh P_1 và P_2 . Hai điểm này cách điểm đỉnh nhọn một khoảng cố định `arrowSize = 14` pixel và bung đối xứng sang hai bên một góc lệch

bệt tương ứng ± 0.45 Radian ngược ra phía sau:

$$x_{P1} = x_{\text{tip}} + 14 \cdot \cos(\theta + \pi - 0.45) \quad (2.9)$$

$$y_{P1} = y_{\text{tip}} + 14 \cdot \sin(\theta + \pi - 0.45) \quad (2.10)$$

$$x_{P2} = x_{\text{tip}} + 14 \cdot \cos(\theta + \pi + 0.45) \quad (2.11)$$

$$y_{P2} = y_{\text{tip}} + 14 \cdot \sin(\theta + \pi + 0.45) \quad (2.12)$$

Sự kết hợp đồng bộ tuần tự từ việc quy hoạch phân bố đa giác đều cho đến kỹ thuật tịnh tiến lượng giác lùi biên đã giúp hệ thống kết xuất thành công mạng lưới đồ thị trực quan động một cách chính xác tuyệt đối.

3. TỔ CHỨC CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

3.1. Phát biểu bài toán hệ thống

- **Đầu vào (Input):** Tập tin lưu trữ ma trận kề kích thước $n \times n$ (với giá trị -1 tượng trưng cho không có liên kết cạnh trực tiếp giữa hai đỉnh), chỉ số đỉnh nguồn u_{start} và đỉnh đích v_{target} .
- **Đầu ra (Output):** Nhật ký thực thi thuật toán kết xuất trực tiếp sang bảng điều khiển đồ họa bên phải của sổ ứng dụng, hình ảnh biểu diễn đồ thị thay đổi trạng thái màu sắc theo diễn tiến thời gian và khung thông báo hiển thị tổng trọng số lộ trình kèm chuỗi đỉnh truy vết logic khi trạng thái chuyển về kết thúc.

3.2. Tổ chức cấu trúc dữ liệu cụ thể

Toàn bộ thông tin đồ thị, tọa độ không gian hình học và nhân tối ưu thuật toán được đóng gói tập trung trong cấu trúc **Graph** định nghĩa như sau:

```
1 #define N 105
2 #define INF 1000000000
3
4 typedef enum {
5     STATE_FIND_MIN,           // Trạng thái tìm kiếm đỉnh có nhân nhỏ nhất
6     STATE_RELAX_EDGES,        // Trạng thái cập nhật tối ưu hóa các cạnh kề
7     STATE_FINISHED            // Trạng thái hoàn tất thuật toán, kết xuất lộ
                                // trình
8 } AlgoState;
9
10 struct Graph {
11     int n;                     // Số lượng đỉnh đồ thị thực tế
12     int A[N][N];               // Ma trận kề lưu trong số cạnh
13     Vector2 toado[N];          // Tọa độ cấu trúc hình học phụ vụ vẽ vòng tròn
                                // đồ họa
14     int L[N];                  // Mảng nhân khoảng cách ngắn nhất tạm thời
15     int T[N];                  // Mảng trạng thái nhân (1: Tạm thời, 0: Có
                                // đỉnh hoàn toán)
16     int par[N];                // Mảng lưu vết phân tử cha để phụ vụ truy vết
                                // đường đi
17 };
```

3.3. Thiết kế máy trạng thái thuật toán mô phỏng

Để đảm bảo giao diện đồ họa tương tác liên tục không bị treo đứng bởi các vòng lặp tính toán sâu, tiến trình Dijkstra được chia nhỏ thành kiến trúc máy trạng thái tích hợp biến kiểm soát thời gian lặp **timer** đồng bộ với tốc độ khung hình thực tế (**GetFrameTime()**). Tiến trình chuyển đổi qua lại mượt mà giữa các bước tìm đỉnh min (**STATE_FIND_MIN**), bước tối ưu hóa cập nhật nhân kề **relax** (**STATE_RELAX_EDGES**) từng cung đơn lẻ và tự động chuyển cấu trúc sang trạng thái kết thúc (**STATE_FINISHED**) khi tìm thấy đích hoặc tập ứng viên rỗng.

4. CHƯƠNG TRÌNH VÀ KẾT QUẢ

4.1. Tổ chức mô hình chương trình

Hệ thống chương trình được phân rã chức năng mạch lạc thành các phân vùng module:

- Khối khởi tạo điều phối giao diện trang bìa Console truyền thống giới thiệu thông tin đề tài học phần.
- Khối xử lý luồng tệp tin dữ liệu đọc cấu trúc đầu vào (`inputdata()`).
- Khối xử lý tính toán hình học không gian đa điểm và vẽ thành phần đồ họa liên kết phối màu (`drawArrow()`).
- Khối động cơ máy trạng thái mô phỏng tương tác chính điều khiển cửa sổ Raylib độ phân giải 1400×900 (`UIRaylib_Dijkstra()`).

4.2. Ngôn ngữ cài đặt

Hệ thống sử dụng ngôn ngữ lập trình C++ chuẩn hóa kiến trúc cấu trúc mảng tối ưu tốc độ kết hợp cùng các thư viện đồ họa mở rộng nền tảng thấp để can thiệp sâu vào bộ đệm kết xuất màn hình trực quan.

4.3. Kết quả thực thi chương trình

4.3.1. Giao diện trang thông tin đề án ban đầu

Khi khởi tạo, ứng dụng hiển thị màn hình giới thiệu định dạng căn lề trung tâm hiển thị đầy đủ thông tin mã hiệu lớp sinh hoạt, tên thành viên nhóm thực hiện và giảng viên hướng dẫn khoa Công nghệ thông tin chuẩn hóa theo cấu trúc của Trường Đại học Bách Khoa.



Hình 4.1: Giao diện chính giới thiệu thông tin đề án trên màn hình Console

4.3.2. Mô phỏng đồ họa động tương tác trực quan

Trong suốt quá trình phần mềm thực thi vòng lặp đồ họa:

- Khung màn hình hiển thị đồ họa phân bố các nút tròn đỉnh đồ thị đều theo cấu trúc vòng elip đồng tâm giúp giảm thiểu tối đa hiện tượng chồng chéo các đường nối cung hướng.
- Nhãn thông tin khoảng cách L cập nhật liên tục dưới chân từng nút tròn. Sắc xanh lá đại diện cho đỉnh đã tối ưu hoàn toàn, sắc vàng biểu thị đỉnh đang đóng vai trò tâm loang xét duyệt hiện tại.

Hình 4.2: Giao diện đồ họa phân bố hệ thống các đỉnh theo quỹ đạo tròn đồng tâm

- Khung bên phải lưu trữ bộ đệm nhật ký thuật toán hiển thị tối đa 20 dòng hoạt động chi tiết, cuộn nội dung nhịp nhàng giúp người học phân tích được chính xác nguyên lý hoạt động nội tại của giải thuật.

Hình 4.3: Bảng điều khiển nhật ký hoạt động cập nhật trạng thái loang nhãn từng bước

- Khi tìm thấy đích, toàn bộ các cạnh và đỉnh cấu thành đường đi ngắn nhất tự động chuyển đổi sắc màu hiển thị sang màu đỏ rực rỡ với độ dày nét vẽ vượt trội nhằm cô lập lộ trình trực quan cho người sử dụng.

Hình 4.4: Biểu diễn lộ trình ngắn nhất bằng hiệu ứng đổi màu cung liên kết sang sắc đỏ

4.3.3. Nhận xét đánh giá hệ thống

- **Ưu điểm:** Ứng dụng giải quyết triệt để bài toán tìm kiếm tối ưu trên mô hình đồ thị có hướng phức tạp. Thiết kế giao diện đồ họa có tính tương tác cao, cơ chế máy trạng thái phân rã logic vận hành trơn tru và trực quan hóa sâu sắc dữ liệu từng bước chạy.
- **Hạn chế tồn tại và điểm nghẽn hiệu năng:**
 - Chương trình hiện tại yêu cầu dữ liệu tệp tin văn bản đầu vào phải tuân thủ nghiêm ngặt cấu trúc định dạng chuẩn hóa sẵn, chưa thiết lập module cảnh báo ngoại lệ khi người dùng cấu hình nhằm ma trận chứa trọng số âm.
 - **Hạn chế của giải thuật:** Về khía cạnh thuật toán cốt lõi, cấu trúc tìm kiếm đỉnh ứng viên có khoảng cách ngắn nhất đang sử dụng vòng lặp duyệt mảng tuyến tính vét cạn liên tục (for chạy từ 1 tới n). Cách làm này tiêu tốn chi phí thời gian là $O(V)$ cho mỗi bước lặp, kéo theo tổng độ phức tạp thời gian toàn cục của chương trình chạm ngưỡng $O(V^2)$ [3]. Đây là một điểm hạn chế lớn khi đồ thị mở rộng về quy mô đỉnh (V lớn) nhưng có mật độ cạnh thưa thớt (E nhỏ).

- **Hướng nâng cấp tối ưu:** Để khắc phục điểm nghẽn hiệu năng trên, cấu trúc mảng duyệt tuần tiến hoàn toàn có thể được thay thế bằng cấu trúc dữ liệu hàng đợi ưu tiên dựa trên cây nhị phân **Min-Heap** tự triển khai thủ công. Việc cải tiến cấu trúc này sẽ hạ chi phí tìm kiếm phần tử nhỏ nhất từ $O(V)$ xuống còn $O(\log V)$, đưa tổng chi phí thời gian thực thi của thuật toán Dijkstra tiệm cận mức tối ưu tuyệt đối là $O((E + V) \log V)$ hoặc $O(E \log V)$.

5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Qua quá trình nghiên cứu và hiện thực hóa báo cáo đồ án học phần PBL1 này, nhóm thực hiện đã làm chủ được kỹ năng xử lý hệ thống ma trận lưu trữ đồ thị liên kết, hiểu sâu bản chất giải thuật loang nhân tối ưu của thuật toán Dijkstra. Đồng thời, việc ứng dụng thành công thư viện đồ họa Raylib đã giúp nhóm tích lũy kinh nghiệm thực tế về lập trình giao diện tương tác, kỹ thuật quản lý luồng sự kiện đồ họa thời gian thực và phương pháp tổ chức mã nguồn khoa học, mạch lạc.

5.2. Hướng phát triển đề tài tương lai

Trong các giai đoạn tiếp theo, nhóm định hướng sẽ nâng cấp ứng dụng với các tính năng mở rộng cao cấp:

- Thiết kế module tương tác chuột linh hoạt cho phép người dùng click chọn trực tiếp trên màn hình để thêm mới đỉnh, kéo thả vẽ cung kết nối và thay đổi trọng số linh hoạt thay vì cấu hình qua tệp tin văn bản tĩnh.
- Nhúng thêm các thuật toán so sánh song song trực quan thời gian thực trên cùng một cấu trúc đồ thị như giải thuật Bellman-Ford hay thuật toán tìm kiếm đường đi định hướng A^* , giúp người dùng dễ dàng nhận biết và đánh giá sự khác biệt về hiệu năng, số lượng bước duyệt nhân giữa các trường phái tư duy giải thuật khác nhau.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Đức Nghĩa, *Giáo trình Toán rời rạc*, Nhà xuất bản Bách Khoa Hà Nội.
- [2] Phan Thanh Tao, *Giáo trình Toán rời rạc*, Trường Đại học Bách Khoa – Đại học Đà Nẵng.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
- [4] Raylib Graph Graphics Development Documentation, <https://www.raylib.com/>.

PHỤ LỤC

Các phân đoạn mã nguồn cốt lõi trong ứng dụng mô phỏng

1. Logic máy trạng thái phân rã bước chạy thuật toán Dijkstra Step-by-Step:

```
1 if (state == STATE_FIND_MIN) {
2     int min_d = INF; currentV = -1;
3     for (int i = 1; i <= g.n; i++)
4         if (g.T[i] && g.L[i] < min_d) { min_d = g.L[i]; currentV = i; }
5
6     if (currentV == -1 || currentV == v_target) {
7         state = STATE_FINISHED;
8         if (g.L[v_target] != INF) {
9             int t = v_target;
10            int temp[N], count = 0;
11            while (t != -1) { onPath[t] = true; temp[count++] = t; t = g
.par[t]; }
12            sprintf(pathResult, "LO TRINH: ");
13            for (int i = count - 1; i >= 0; i--) {
14                char b[15]; sprintf(b, "%d%s", temp[i], (i == 0 ? "" : "
-> "));
15                strcat(pathResult, b);
16            }
17            } else sprintf(pathResult, "KHONG TIM THAY DUONG DI");
18            AddLog(">>> KET THUC: Da xac dinh lo trinh.", stepLogs, &
logCount);
19        } else {
20            g.T[currentV] = 0; neighborIdx = 1; state = STATE_RELAX_EDGES;
21            AddLog(TextFormat("Chon dinh %d (L=%d) nho nhatt", currentV, g.L[
currentV])), stepLogs, &logCount);
22        }
23    }
24    else if (state == STATE_RELAX_EDGES) {
25        bool found = false;
26        while (neighborIdx <= g.n) {
27            if (g.A[currentV][neighborIdx] != -1 && g.T[neighborIdx]) {
28                int newD = g.L[currentV] + g.A[currentV][neighborIdx];
29                if (newD < g.L[neighborIdx]) {
30                    g.L[neighborIdx] = newD;
31                    g.par[neighborIdx] = currentV;
32                    AddLog(TextFormat(" + Toi uu L[%d] = %d", neighborIdx,
newD), stepLogs, &logCount);
33                } else AddLog(TextFormat(" - Canh %d->%d khong toi uu",
currentV, neighborIdx), stepLogs, &logCount);
34                neighborIdx++; found = true; break;
35            }
36            neighborIdx++;
37        }
38        if (!found) state = STATE_FIND_MIN;
39    }
```

2. Hàm xử lý vẽ mũi tên định hướng và hiển thị trọng số cạnh:


```

1 void drawArrow(Vector2 start, Vector2 end, float rDinh, Color mau, int
   trongSo, float thickness) {
2     float angle = atan2f(end.y - start.y, end.x - start.x);
3     Vector2 tip = { end.x - (rDinh + 5) * cosf(angle), end.y - (rDinh +
   5) * sinf(angle) };
4     DrawLineEx(start, tip, thickness, mau);
5
6     float arrowSize = 14.0f;
7     Vector2 p1 = { tip.x + arrowSize * cosf(angle + PI - 0.45f), tip.y +
   arrowSize * sinf(angle + PI - 0.45f) };
8     Vector2 p2 = { tip.x + arrowSize * cosf(angle + PI + 0.45f), tip.y +
   arrowSize * sinf(angle + PI + 0.45f) };
9     DrawTriangle(tip, p2, p1, mau);
10
11     if (trongSo != -1) {
12         char buf[10]; sprintf(buf, "%d", trongSo);
13         Vector2 mid = { (start.x + tip.x) / 2, (start.y + tip.y) / 2 };
14         DrawRectangle(mid.x - 2, mid.y - 2, MeasureText(buf, 20) + 8,
   24, Fade(RAYWHITE, 0.7f));
15         DrawText(buf, mid.x + 2, mid.y, 20, (mau.r > 200 && mau.g < 100)
   ? RED : DARKGRAY);
16     }
17 }

```